SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Dry Run Evaluation

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO1 – Apply (BL3)	**Assessment:**	Assessment Tool 2 – Dry Run Evaluation
Weightage:	35% of CIA	**Duration / Marks:**	60 mins / 100 marks
CO1 Statement:	Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)		
Student Name:	K. Madhulika	**Reg. No.:**	192524190
Section / Batch:	Slot A	**Date:**	10-10-2026

Code of Conduct

I _K. Madhulika, 192524190_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __Madhulika_____

Instructions

- Read each question carefully; answer all four questions (25 marks each, 100 in total).
- Show every dry-run step: the initial queue, stack, priority queue or belief state, each iteration, your assumptions and a clear final answer, and finish with the interpretation asked for.
- Follow the conventions stated in each question (alphabetical expansion, explored set, goal test on removal from the frontier, tie-breaking) and present tables neatly.
- Marks follow the Dry Run Evaluation rubric: understanding of the search problem, state expansion and dry run execution, application of the search algorithm, accuracy of the solution path, and presentation and logical reasoning.
- Upload in GitHub and GCR (question paper with Reg. No., Name, answers, rubrics and evaluation page at the end). Duration: 1 hour (15 minutes per question).

Section / Topic	Focus	Question Nos.
1. Foundations and History of AI	Four approaches to AI, Turing Test, eras and milestones	**Q2**
2. Intelligent Agents: Agents and Environments	Agent, percepts, actions, agent function and program, PEAS	**Q1**
3. The Concept of Rationality	Performance measure, percept sequence, rational vs omniscient	**Q1**
4. The Nature of Environments	Task-environment properties and their classification	**Q1**
5. The Structure of Agents	Simple reflex and model-based reflex agent programs	**Q1**
6. Problem-Solving Agents	Goal formulation, problem formulation, search, execution	**Q2**
7. Problem Formulation	States, initial state, actions, goal test, path cost; belief states	**Q2, Q4**
8. Searching for Solutions	Search graph, frontier, explored set, node expansion	**Q3**
9. Uninformed Search Strategies	Dry runs of BFS, DFS and uniform-	**Q3**

	cost search	
10. Search with Partial Information	Sensorless problems, belief states, percept update	**Q4**

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

Topic: Intelligent agents, rationality, nature of environments, structure of agents | 15 minutes

A vacuum-cleaner agent lives in a world with two squares, A and B. At each time step it perceives [its location, status of its own square] and performs one action: Suck, Right, Left or NoOp. The agent starts in square A, both squares are initially Dirty, and dirt never reappears.

Performance measure over T = 6 time steps: +10 for every square cleaned (a Suck on a Dirty square) and −1 for every Left or Right move.

Agent X (simple reflex): if the current square is Dirty, Suck; otherwise move to the other square.

Agent Y (model-based reflex): keeps an internal state of the squares it knows to be clean. If the current square is Dirty, Suck; otherwise, if both squares are known clean, NoOp; otherwise move to the other square.

(a) Write the PEAS description (Performance measure, Environment, Actuators, Sensors) of this agent. **[4 Marks]**

(b) Classify the task environment on observability, number of agents, determinism, episodic/sequential, static/dynamic and discrete/continuous, with a one-phrase justification each. **[4 Marks]**

(c) Dry-run both agents for all 6 time steps. Complete the table (percept, action, and Y's internal state after the step). **[9 Marks]**

Step	X: Percept	X: Action	Y: Percept	Y: Action	Y: Known-Clean Squares
1					
2					
3					
4					
5					
6					

(d) Compute the total performance of X and of Y over the 6 steps. **[3 Marks]**

(e) Interpret: which agent is rational? Both agents receive the identical percept [B, Clean] at step 4 – explain why they act differently, using the structure of the two agent programs, and distinguish a rational agent from an omniscient agent. **[5 Marks]**

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Topic: Foundations and history of AI, problem-solving agents, problem formulation | 15 minutes

Part A. Complete the table by classifying each system or idea under one of the four approaches to AI (thinking humanly, acting humanly, thinking rationally, acting rationally) and naming the era or milestone of AI history to which it belongs.

(a) Complete the table. **[8 Marks]**

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)		
General Problem Solver (Newell and Simon, 1957)		
Aristotle's syllogisms and formal logic		
An agent that does the right thing to maximise expected performance		

Part B. A problem-solving agent has two jugs: X of capacity 4 litres and Y of capacity 3 litres, both initially empty and without measuring marks. It may fill a jug from a tap, empty a jug on the ground, or pour one jug into the other until the source is empty or the target is full. The goal is to have exactly 2 litres in jug X.

(b) Formulate the problem: state representation, initial state, actions, transition model, goal test and path cost. **[6 Marks]**

(c) Generate all successor states of (0, 0) and of (4, 0) (leave out actions that leave the state unchanged). **[3 Marks]**

(d) How many states does the state space contain? Write a shortest solution as a sequence of states and give its path cost. **[3 Marks]**

(e) Interpret: describe the four phases (goal formulation, problem formulation, search, execution) of a problem-solving agent for this task, and explain why the agent can execute the found solution without looking at new percepts. **[5 Marks]**

Question 3: Searching for Solutions – Uninformed Search Strategies

Topic: Searching for solutions, BFS, DFS, uniform-cost search | 15 minutes

Consider the weighted undirected graph below. The start node is S and the goal node is G. Use graph search with an explored set; expand neighbours alphabetically, test the goal when a node leaves the frontier, and break ties alphabetically. BFS and DFS ignore edge costs while searching, but path costs are measured with them.

Edge	Cost	Edge	Cost
S – A	1	B – D	1
S – B	4	C – G	6
A – C	2	D – G	2
A – D	5		

(a) Perform BFS from S (queue initially [S]). Do not add a node that is already explored or already in the queue. **[6 Marks]**

Iteration	Node Dequeued	Queue after Expansion	Explored Set
1			
2			
3			
4			
5			
6			

(b) Perform DFS from S using a stack (initially [S]); push children so that the alphabetically first child is popped first. Write the stack with its top on the left. **[5 Marks]**

Iteration	Node Popped	Stack after Pushing Children
1		
2		
3		
4		

(c) Perform uniform-cost search from S. Write queue entries as Node:g(parent) and replace an entry whenever a cheaper path to a frontier node is found. **[7 Marks]**

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
1			
2			
3			
4			

Iteration	Node Expanded	g(n)	Priority Queue after Expansion
5			
6			

(d) Complete the comparison table. **[4 Marks]**

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS				
DFS				
UCS				

(e) Interpret: explain why UCS removed as many nodes as BFS yet found a cheaper path, why DFS needed the fewest nodes here but cannot be trusted in general, and why an explored set is needed in graph search (use iteration 3 of BFS as an example). **[3 Marks]**

Question 4: Search with Partial Information – Belief States

Topic: Problem formulation, search with partial information (sensorless and percept-based belief-state search) | 15 minutes

A vacuum agent lives in a two-square world (Left square, Right square), each square being Clean (C) or Dirty (D). Actions: Left, Right and Suck (Left in the left square and Right in the right square leave the agent where it is; Suck cleans the current square). Physical states are written (Agent, Left square, Right square):

State	Description	State	Description
S1	(L, D, D)	S5	(L, C, D)
S2	(R, D, D)	S6	(R, C, D)
S3	(L, D, C)	S7	(L, C, C)
S4	(R, D, C)	S8	(R, C, C)

(a) Initially the agent has no sensors and does not know its location or the dirt status. Formulate the problem over belief states: initial belief state, actions, goal test and path cost. **[3 Marks]**

(b) Apply the action sequence [Right, Suck, Left, Suck] and write the belief state after each action. **[6 Marks]**

Step	Action	Belief State after Action
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}
1	Right	
2	Suck	
3	Left	
4	Suck	

(c) Now the agent has a local sensor that reports [location, status of the current square]. Starting again from the initial belief state, the first percept is [Left, Dirty]; the agent then performs Suck and perceives [Left, Clean]; it then performs Right and perceives [Right, Dirty]. Fill in the belief state after each action (prediction) and after each percept (update). **[8 Marks]**

Step	Action	After Action (Predicted)	Percept	After Percept (Updated)
0	(start)	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	
1	Suck		[Left, Clean]	
2	Right		[Right, Dirty]	

(d) How many different belief states are possible when there are 8 physical states? Why does this make the problem harder than searching over physical states? **[3 Marks]**

(e) Interpret: explain why the percepts in part (c) shrink the belief state, how this differs from the sensorless case in part (b), and why the sensorless plan can still be called rational. **[5 Marks]**

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Accuracy of Solution Path	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Q. No. / Criteria	Understanding of Search Problem	State Expansion and Dry Run Execution	Application of Search Algorithm	Accuracy of Solution Path	Accuracy of Solution Path	Sub. Total (20)	Total %
1							
2							
3							
4							
Total							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO1 AT2 – Dry Run Evaluation

Name: K. Madhulika

Reg no: 1925524190

CSA 1716 – Artificial Intelligence

Question 1: Agents, Environments and Rationality – Dry Run of Agent Programs

(a) PEAS Description

PEAS stands for Performance Measure, Environment, Actuators, and Sensors. It describes the task environment in which an intelligent agent operates.

Component	Description
P – Performance Measure	+10 points for each square cleaned by sucking dirt and −1 point for every Left or Right movement over 6 time steps.
E – Environment	Two squares, A and B. Both squares are initially dirty. Dirt never reappears.
A – Actuators	Suck, Right, Left, and NoOp (do nothing).
S – Sensors	The agent perceives its current location (A or B) and the status of its current square (Dirty or Clean).

Explanation: The vacuum-cleaner agent uses its sensors to identify its location and the dirt status of its current square. Based on the percept, it selects an action to clean the squares while minimizing unnecessary movements.

(b) Classification of the Task Environment

The vacuum-cleaner world can be classified as follows:

Property	Classification	Justification
Observability	Partially observable	The agent perceives only its current square and cannot directly observe the other square's status.

Number of agents	Single-agent	Only one vacuum-cleaner agent operates in the environment.
Determinism	Deterministic	Each action has a predictable outcome; for example, Suck cleans a dirty square.
Episodic or Sequential	Sequential	Each action can affect future decisions and the agent's subsequent location.
Static or Dynamic	Static	The dirt status does not change on its own, and dirt never reappears.
Discrete or Continuous	Discrete	The locations, percepts, and available actions are finite and distinct.

Conclusion: The vacuum-cleaner environment is partially observable, single-agent, deterministic, sequential, static, and discrete.

(c) Dry Run of Agent X and Agent Y

1. Initial Conditions and Assumptions

The initial environment is:

Square	**Initial Status**
A	Dirty
B	Dirty

Both agents start at square A.

Initial state:

- Agent X's location: A
- Agent Y's location: A
- Agent Y's known-clean set: {} (empty)
- Time steps: 6
- Dirt never reappears.

2. Agent Programs

Agent X – Simple Reflex Agent

1. If the current square is Dirty, perform Suck.
2. Otherwise, move to the other square.

Agent X does not remember the cleanliness of previously visited squares.

Agent Y – Model-Based Reflex Agent

1. If the current square is Dirty, perform Suck.
2. Otherwise, if both squares are known to be clean, perform NoOp.
3. Otherwise, move to the other square.

Agent Y maintains an internal state containing the squares it knows to be clean.

3. Dry Run of Agent X

Step	Location Before Action	Percept	Action	Location After Action	Environment After Action
1	A	[A, Dirty]	Suck	A	A Clean, B Dirty
2	A	[A, Clean]	Right	B	A Clean, B Dirty
3	B	[B, Dirty]	Suck	B	A Clean, B Clean
4	B	[B, Clean]	Left	A	A Clean, B Clean
5	A	[A, Clean]	Right	B	A Clean, B Clean
6	B	[B, Clean]	Left	A	A Clean, B Clean

Observation: Agent X continues moving between A and B even after both squares have been cleaned because it has no memory of the other square's cleanliness.

4. Dry Run of Agent Y

For Agent Y, the known-clean set is updated whenever the agent perceives a square as Clean or successfully cleans it with Suck.

For this dry run, the internal state is updated after the action at each time step.

Step	Location Before Action	Percept	Action	Known-Clean Squares After Step
1	A	[A, Dirty]	Suck	{A}
2	A	[A, Clean]	Right	{A}
3	B	[B, Dirty]	Suck	{A, B}
4	B	[B, Clean]	NoOp	{A, B}
5	B	[B, Clean]	NoOp	{A, B}
6	B	[B, Clean]	NoOp	{A, B}

Observation: Agent Y remembers that both squares are clean. Therefore, it performs NoOp instead of continuing to move unnecessarily.

5. Combined Dry-Run Table

The following table follows the format specified in the question.

Step	Percept X	Action X	Percept Y	Action Y	Y's Known-Clean Squares After Step
1	[A, Dirty]	Suck	[A, Dirty]	Suck	{A}
2	[A, Clean]	Right	[A, Clean]	Right	{A}
3	[B, Dirty]	Suck	[B, Dirty]	Suck	{A, B}
4	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}
5	[A, Clean]	Right	[B, Clean]	NoOp	{A, B}
6	[B, Clean]	Left	[B, Clean]	NoOp	{A, B}

Important clarification: The combined table shows that at step 4, both agents receive the identical percept [B, Clean], but their actions differ because their agent programs use different information.

Agent X follows the rule “otherwise move to the other square,” whereas Agent Y checks its internal state and recognizes that both squares are already clean.

(d) Calculation of Total Performance

The performance measure over six time steps is:

- +10 points for each square cleaned by a Suck action when it is Dirty.
- −1 point for each Left or Right movement.
- 0 points for NoOp.

The formula is:

Total Performance=10C−M

Where:

- CC = Number of squares cleaned by a successful Suck action.
- MM = Number of Left or Right movements.

Agent X

Successful cleaning actions:

- Step 1: Suck on A = +10
- Step 3: Suck on B = +10

Total cleaning score:

10×2=20

Movement actions:

- Step 2: Right = −1
- Step 4: Left = −1
- Step 5: Right = −1
- Step 6: Left = −1

Total movement penalty:

4× (−1) =−44

Therefore,

Performance of X=20−4=16

Agent Y

Successful cleaning actions:

- Step 1: Suck on A = +10
- Step 3: Suck on B = +10

Total cleaning score:

10×2=20

Movement actions:

- Step 2: Right = −1

Total movement penalty:

1× (−1) =−11

NoOp actions:

- Steps 4, 5, and 6 = 0 points.

Therefore,

Performance of Y=20−1=19

Performance Comparison

Agent	Cleaning Score	Movement Penalty	Total Performance
Agent X	+20	−4	**16**
Agent Y	+20	−1	**19**

Result: Agent Y achieves a higher performance score than Agent X by 3 points because it avoids unnecessary movements after both squares are known to be clean.

(e) Interpretation: Rationality and Agent Structure

1. Which Agent Is Rational?

Agent Y is more rational than Agent X in this scenario because it achieves the same cleaning result while making fewer movements and obtaining a higher performance score.

Agent Y uses an internal state to remember which squares are clean. Once both squares are known to be clean, it performs NoOp, avoiding unnecessary movement penalties.

Agent X does not maintain an internal state. It repeatedly moves between squares whenever the current square is clean, even when both squares have already been cleaned.

2. Why Do the Agents Act Differently at Step 4?

At step 4, both agents receive the same percept:

[B, Clean]

However, their actions differ because their programs use different information.

Feature	**Agent X – Simple Reflex**	**Agent Y – Model-Based Reflex**
Current percept	[B, Clean]	[B, Clean]
Internal state	Does not maintain memory of the other square	Remembers that both A and B are clean
Decision rule	If the current square is clean, move to the other square	If the current square is clean and both squares are known clean, perform NoOp
Action at step 4	Left	NoOp
Reason	Follows a rule based only on the current percept	Uses the current percept and internal state

Explanation: A simple reflex agent selects an action using only the current percept. A model-based reflex agent maintains an internal representation of the environment and uses it along with the current percept to select an action.

Thus, identical percepts can lead to different actions when the agents have different internal states or decision rules.

3. Rational Agent vs. Omniscient Agent

A rational agent selects the action expected to maximize its performance measure based on its percept history, available knowledge, and possible actions.

An omniscient agent would know the actual consequences of every possible action and the true state of the environment, including information that may not be available to an ordinary agent.

The differences are:

Rational Agent	**Omniscient Agent**
Chooses the best expected action using available information.	Knows the actual outcomes of actions in advance.
May make mistakes because information is limited.	Has complete knowledge of the relevant facts and outcomes.
Does not need perfect knowledge to be rational.	Has knowledge beyond what an ordinary agent can obtain through its sensors.

Conclusion: Rationality does not require omniscience. An agent can be rational even if it cannot know everything about the environment. It must choose the best action based on the information available to it.

Final Conclusion

The vacuum-cleaner problem demonstrates the difference between simple reflex and model-based reflex agents.

- Agent X reacts only to the current percept and continues moving even after cleaning both squares.
- Agent Y maintains an internal state and recognizes when both squares are clean.
- Both agents clean both squares and earn 20 points from successful cleaning actions.
- Agent X scores 16, while Agent Y scores 19 over sixtime steps.
- Agent Y performs better because it avoids unnecessary movements.

Therefore, Agent Y is the better-performing agent in this scenario, illustrating how internal state and environmental knowledge can improve an agent's decisions.

Question 2: Foundations and History of AI; Problem-Solving Agent and Problem Formulation

Foundations and History of AI

Artificial Intelligence (AI) is a branch of computer science that focuses on developing systems capable of performing tasks that normally require human intelligence, such as reasoning, learning, problem-solving, and decision-making.

There are four approaches to AI:

1. **Thinking Humanly:** Developing systems that imitate human thought processes.
2. **Acting Humanly:** Developing systems that behave like humans.
3. **Thinking Rationally:** Developing systems that reason logically.
4. **Acting Rationally:** Developing systems that select actions to achieve the best expected outcomes.

(a) Completed Table

System / Idea	Approach to AI	Era / Milestone
Turing Test proposed (1950)	Acting Humanly	Early AI history; 1950, Alan Turing's proposal for evaluating machine intelligence
General Problem Solver (Newell and Simon, 1957)	Thinking Humanly	Early AI era; cognitive simulation and symbolic problem-solving research
Aristotle's syllogisms and formal logic	Thinking Rationally	Ancient Greek philosophy; foundations of logical reasoning
An agent that does the right thing to maximise expected performance	Acting Rationally	Rational-agent approach; modern AI framework

Explanation

1. Turing Test (1950)

The Turing Test was proposed by Alan Turing to evaluate whether a machine could exhibit behaviour indistinguishable from that of a human in a text-based conversation.

It belongs to the acting humanly approach because it evaluates observable behaviour rather than requiring the machine to think exactly like a human.

2. General Problem Solver (1957)

The General Problem Solver was developed by Allen Newell and Herbert A. Simon, with Cliff Shaw also contributing to the project.

It attempted to solve problems using symbolic reasoning and general problem-solving techniques. It is associated with the study of human problem-solving and cognitive processes, making thinking humanly the most suitable classification among the four approaches.

3. Aristotle's Syllogisms and Formal Logic

Aristotle developed influential principles of logical reasoning, including syllogistic logic.

For example:

- All humans are mortal.
- Socrates is a human.
- Therefore, Socrates is mortal.

This illustrates the thinking rationally approach because the conclusion follows logically from the premises.

4. Rational Agent

A rational agent selects actions that maximize expected performance based on its percepts, knowledge, and available actions.

For example, a vacuum-cleaner agent that remembers which squares are clean and avoids unnecessary movements is using information to make rational decisions.

This belongs to the acting rationally approach.

Problem Formulation — Two-Jug Problem

Problem Statement

A problem-solving agent has two water jugs:

- Jug X has a capacity of 4 litres.
- Jug Y has a capacity of 3 litres.
- Both jugs are initially empty.
- Neither jug has measuring marks.
- The agent can fill, empty, and pour water between the jugs.
- The goal is to obtain exactly 2 litres in Jug X.

(b) Formulation of the Problem

A problem-solving agent represents the problem using six components.

Component	Description
1.State Representation	A state is represented as (x, y), where x is the amount of water in Jug X and y is the amount of water in Jug Y. Here, 0≤x≤40 \leq x \leq 4 and 0≤y≤30 \leq y \leq 3.
2. Initial State	(0, 0) — both jugs are empty.
3. Actions	Fill X, Fill Y, Empty X, Empty Y, Pour X into Y, and Pour Y into X.
4.Transition Model	Describes the resulting state after an action, respecting the jug capacities and the amount of water available.
5. Goal Test	Check whether the amount of water in Jug X is exactly 2 litres: x=2x=2.
6. Path Cost	Assume each valid action costs 1. The total path cost is the number of actions taken.

Transition Model

The actions are defined as follows.

1. Fill X

Fill Jug X to its capacity of 4 litres.

(x, y) → (4, y)

2. Fill Y

Fill Jug Y to its capacity of 3 litres.

$(x, y) \rightarrow (x, 3)$

3. Empty X

Empty Jug X onto the ground.

$(x, y) \rightarrow (0, y)$

4. Empty Y

Empty Jug Y onto the ground.

$(x, y) \rightarrow (x, 0)$

5. Pour X into Y

Transfer water from X to Y until X is empty or Y is full.

$t = \min(x, 3-y)$

The resulting state is:

$(x, y) \rightarrow (x-t, y+t)$

6. Pour Y into X

Transfer water from Y to X until Y is empty or X is full.

$t = \min(y, 4-x)$

The resulting state is:

$(x, y) \rightarrow (x+t, y-t)$

These rules ensure that no jug exceeds its capacity and that pouring conserves the total amount of water.

Part C: Generate Successor States

A successor state is a state that can be reached from the current state by performing one valid action.

We must generate all successor states for (0, 0) and (4, 0), excluding actions that leave the state unchanged.

1. Successors of (0, 0)

Initial state:

(0,0)

Both jugs are empty.

Action	**Resulting State**	**Explanation**
Fill X	(4, 0)	Jug X is filled to 4 litres.
Fill Y	(0, 3)	Jug Y is filled to 3 litres.
Empty X	(0, 0)	No change; excluded.
Empty Y	(0, 0)	No change; excluded.
Pour X into Y	(0, 0)	No water in X; excluded.
Pour Y into X	(0, 0)	No water in Y; excluded.

Successor set:

Succ f_0 (0,0)={(4,0),(0,3)}

Therefore, there are 2 distinct successor states.

2. Successors of (4, 0)

Current state:

(4,0)

Jug X contains 4 litres, and Jug Y is empty.

Action	**Resulting State**	**Explanation**
Fill X	(4, 0)	X is already full; no change.
Fill Y	(4, 3)	Fill Y to 3 litres.
Empty X	(0, 0)	Empty all 4 litres from X.

Empty Y	(4, 0)	Y is already empty; no change.
Pour X into Y	(1, 3)	Transfer 3 litres from X to Y; X retains 1 litre.
Pour Y into X	(4, 0)	Y is empty; no change.

Successor set: Succ $_{f0}$ (4,0) = {(4,3), (0,0), (1,3)}

Therefore, there are 3 distinct successor states.

State-Space Size and Shortest Solution

1. Number of States in the State Space

Jug X can contain any whole-number quantity from 0 to 4 litres:

X= {0,1,2,3,4}

Therefore, Jug X has 5 possible amounts.

Jug Y can contain any whole-number quantity from 0 to 3 litres:

Y= {0,1,2,3}

Therefore, Jug Y has 4 possible amounts.

The total number of possible states is:

|S|= (4+1) (3+1)

Assumption: As is conventional in the discrete water-jug problem, the state space uses whole-litre quantities. If arbitrary continuous water quantities were permitted, the state space would not contain only 20 states.

2. Finding the Shortest Solution

We use Breadth-First Search (BFS) because every action has an equal cost of 1. BFS explores states level by level and finds a shortest solution in terms of the number of actions.

The goal test is performed when a state is removed from the frontier, as specified in the question's general dry-run instructions.

Initial state:

(0,0)

Goal condition:

x=2

BFS Dry Run

For this dry run:

- Successors are generated in the order: Fill X, Fill Y, Empty X, Empty Y, Pour X into Y, Pour Y into X.
- Repeated states are not added to the frontier if they have already been explored or are already waiting in the queue.
- The goal test is performed when a state is removed from the queue.

The queue is shown from front to rear.

Iteration	**State Removed**	**Successor States Added**	**Queue After Expansion**
Initial	—	—	[(0,0)]
1	(0,0)	(4,0), (0,3)	[(4,0), (0,3)]
2	(4,0)	(4,3), (1,3)	[(0,3), (4,3), (1,3)]
3	(0,3)	(4,3), (0,0); both already discovered	[(4,3), (1,3)]
4	(4,3)	(0,3), (4,0); both already discovered	[(1,3)]
5	(1,3)	(1,0), (4,0), (0,3), (4,0); discard repeated states	[(1,0)]
6	(1,0)	(0,0), (0,1), (4,0); discard repeated states	[(0,1)]
7	(0,1)	(4,1), (0,0); discard repeated state	[(4,1)]
8	(4,1)	(4,0), (0,1), (2,3)	[(2,3)]
9	(2,3)	Goal test succeeds because x=2x=2	Stop

Important: The table above contains a mistake in the iteration 8 transition if interpreted using only the listed successors: pouring X into Y from (4,1) produces (2,3), which is valid. Thus, the goal is reached.

3. Shortest Solution as a Sequence of States

The solution path found by BFS is:

(0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3)

4. Action Sequence

Step	State	Action Performed
0	(0,0)	Initial state
1	(4,0)	Fill Jug X
2	(1,3)	Pour X into Y
3	(1,0)	Empty Jug Y
4	(0,1)	Pour the remaining 1 litre from X into Y?

Correction to the sequence: The state (1,0) means X contains 1 litre and Y is empty. Pouring X into Y produces (0,1), so the action in Step 4 should be Pour X into Y. The subsequent steps are then valid.

Step	State	Action Performed
4	(0,1)	Pour X into Y
5	(4,1)	Fill Jug X
6	(2,3)	Pour X into Y until Y is full

The goal condition is satisfied because Jug X contains exactly 2 litres.

5. Path Cost

Each action has a cost of 1.

The solution requires 6 actions:

1. Fill X
2. Pour X into Y
3. Empty Y
4. Pour X into Y
5. Fill X
6. Pour X into Y

Therefore,

Path Cost=6

Final result: The shortest solution contains 6 actions and reaches the state (2,3), where Jug X contains exactly 2 litres.

Four Phases of a Problem-Solving Agent

A problem-solving agent follows four major phases: goal formulation, problem formulation, search, and execution.

1. Goal Formulation

The agent first decides what it wants to achieve.

In this problem, the goal is to obtain exactly 2 litres of water in Jug X.

The goal condition is:

x=2

Jug Y may contain any quantity between 0 and 3 litres.

2. Problem Formulation

The agent represents the problem in a form that can be solved using a search algorithm.

It identifies:

- **Initial state:** (0,0)
- **Possible actions:** Fill, empty, and pour.
- **Transition model:** The resulting state after each action.

- **Goal test:** Check whether x=2.
- **Path cost:** Each action costs 1.

This representation allows the agent to explore possible solutions systematically.

3. Search

The agent uses a search algorithm to find a sequence of actions that leads from the initial state to a goal state.

In this problem, Breadth-First Search (BFS) is suitable because every action has the same cost.

BFS explores states in increasing order of depth and finds a solution with the minimum number of actions.

The resulting action sequence is:

Fill X → Pour X into Y → Empty Y → Pour X into Y → Fill X → Pour X into Y.

The final state is (2,3).

4. Execution

After finding the solution, the agent executes the actions in the planned order.

Step	Action	Resulting State
1	Fill X	(4,0)
2	Pour X into Y	(1,3)
3	Empty Y	(1,0)
4	Pour X into Y	(0,1)
5	Fill X	(4,1)
6	Pour X into Y	(2,3)

After the sixth action, the agent checks the goal condition and confirms that Jug X contains exactly 2 litres.

Why Can the Agent Execute the Solution Without Looking at New Percepts?

The agent can execute the plan without obtaining new percepts at every step because the problem is assumed to have a fully observable, deterministic environment.

- **Fully observable:** The amounts of water in both jugs are known and represented by the current state.
- **Deterministic:** Each action produces a predictable result.
- **Known transition model:** The agent knows how filling, emptying, and pouring change the amounts of water.

Therefore, once the agent finds the solution, it can predict the result of each action and execute the sequence without repeatedly observing the environment.

Conclusion: The two-jug problem illustrates how a problem-solving agent represents a problem as states and actions, explores possible paths using BFS, and executes a shortest sequence of actions to reach the desired goal.

Question 3: Searching for Solutions – Uninformed Search Strategies

Given Graph

The graph is an undirected, weighted graph with start node **S** and goal node **G**.

Edges and Costs

Edge	Cost
S – A	1
S – B	4
A – C	2
A – D	5
B – D	1
C – G	6
D – G	2

Given conditions:

- Start node: S
- Goal node: G
- Search type: Graph search with an explored set
- Neighbours are expanded alphabetically.
- Goal testing occurs when a node is removed from the frontier.
- BFS and DFS ignore edge costs while searching, but their solution path costs are calculated using edge weights.
- UCS uses cumulative path cost g(n)g(n).
- For DFS, the stack's top is shown on the left.

(a) Breadth-First Search (BFS)

Definition

Breadth-First Search explores nodes level by level. It uses a FIFO (First In, First Out) queue.

The algorithm explores all nodes at the current depth before exploring nodes at the next depth.

Initial Conditions

- Queue: [S]
- Explored set: {}

Step-by-Step Dry Run

Iteration 1

Remove S from the queue.

Neighbours of S in alphabetical order: A, B.

Add A and B to the queue.

- Node dequeued: S
- Queue after expansion: [A, B]
- Explored set: {S}

Iteration 2

Remove A from the queue.

Neighbours of A in alphabetical order: C, D, S.

- C and D are added.
- S is already explored, so it is not added.
- Node dequeued: A
- Queue after expansion: [B, C, D]
- Explored set: {S, A}

Iteration 3

Remove B from the queue.

Neighbours of B in alphabetical order: D, S.

- D is already in the queue, so it is not added again.
- S is already explored.
- Node dequeued: B
- Queue after expansion: [C, D]
- Explored set: {S, A, B}

Iteration 4

Remove C from the queue.

Neighbours of C in alphabetical order: A, G.

- A is already explored.
- G is added to the queue.
- Node dequeued: C
- Queue after expansion: [D, G]
- Explored set: {S, A, B, C}

Iteration 5

Remove D from the queue.

Neighbours of D in alphabetical order: A, B, G.

- A and B are already explored.
- G is already in the queue, so it is not added again.
- Node dequeued: D
- Queue after expansion: [G]
- Explored set: {S, A, B, C, D}

Iteration 6

Remove G from the queue.

The goal test succeeds because G is the goal node.

Stop the search.

- Node dequeued: G
- Queue after expansion: []
- Explored set: {S, A, B, C, D, G}

Completed BFS Table

Iteration	Node Dequeued	Queue After Expansion	Explored Set
1	S	[A, B]	{S}
2	A	[B, C, D]	{S, A}
3	B	[C, D]	{S, A, B}
4	C	[D, G]	{S, A, B, C}
5	D	[G]	{S, A, B, C, D}
6	G	[]	{S, A, B, C, D, G}

BFS Solution Path

BFS finds the path with the fewest edges.

S→A→C→G

Path cost:

1+2+6=9

Result: BFS removes 6 nodes from the frontier, including the goal node.

(b) Depth-First Search (DFS)

Definition

Depth-First Search explores one branch as deeply as possible before backtracking.

It uses a LIFO (Last In, First Out) stack.

The question specifies that the alphabetically first child must be popped first. Therefore, when pushing children, we push them in reverse alphabetical order.

Initial Conditions

- Stack: [S]
- Explored set: {}

The **top of the stack is on the left**.

Step-by-Step Dry Run

Iteration 1

Pop S.

Its neighbours are A and B. Push B first, then A, so A is on top.

- Node popped: S
- Stack after pushing children: [A, B]
- Explored set: {S}

Iteration 2

Pop A.

Its neighbours are C, D, S.

- S is already explored.
- Push D, then C, so C is on top.
- Node popped: A
- Stack after pushing children: [C, D, B]
- Explored set: {S, A}

Iteration 3

Pop C.

Its neighbours are A and G.

- A is already explored.
- Push G.
- Node popped: C
- Stack after pushing children: [G, D, B]
- Explored set: {S, A, C}

Iteration 4

Pop G.

The goal test succeeds.

Stop the search.

- Node popped: G
- Stack after expansion: [D, B]
- Explored set: {S, A, C, G}

Completed DFS Table

Iteration	Node Popped	Stack After Pushing Children (Top Left)
1	S	[A, B]

2	A	[C, D, B]
3	C	[G, D, B]
4	G	[D, B]

DFS Solution Path

S→A→C→G

Path cost:

1+2+6=9

Result: DFS removes 4 nodes from the frontier, including the goal node.

Note: The stack table follows the common graph-search convention that nodes already explored are not pushed again. The goal is tested when removed from the stack.

(c) Uniform-Cost Search (UCS)

Definition

Uniform-Cost Search expands the frontier node with the lowest cumulative path cost g(n).

Unlike BFS and DFS, UCS considers edge weights when deciding which node to expand.

For each frontier node, we maintain:

- Node name
- Cumulative path cost g(n)g(n)
- Parent node

The queue is ordered by increasing cumulative cost. Alphabetical order breaks ties.

Initial Conditions

- Priority queue: [S:0(-)]
- Explored set: {}
- Start-node cost: g(S)=0

Notation: A:1(S) means node A has cumulative cost 1 and parent S.

Step-by-Step Dry Run

Iteration 1

Remove S with cost 0.

Neighbours:

- A: 0+1=1
- B: 0+4=4

Priority queue:

[A:1(S), B:4(S)]

Explored set: {S}

Iteration 2

Remove A with cost 1.

Neighbours:

- C: 1+2=3
- D: 1+5=6
- S: 1+1=2, but S is already explored.

Priority queue:

[C:3(A), B:4(S), D:6(A)]

Explored set: {S, A}

Iteration 3

Remove C with cost 3.

Neighbours:

- A: 3+2=5, but A is already explored.
- G: 3+6=9

Add G with cost 9.

Priority queue:

[B:4(S), D:6(A), G:9(C)]

Explored set: {S, A, C}

Iteration 4

Remove B with cost 4.

Neighbours:

- D: 4+1=5
- S: 4+4=8, but S is already explored.

The new path to D costs 5, which is cheaper than the existing frontier cost of 6.

Therefore, replace D:6(A) with D:5(B).

Priority queue:

[D:5(B), G:9(C)]

Explored set: {S, A, C, B}

Iteration 5

Remove D with cost 5.

Neighbours:

- A: 5+5=10, but A is already explored.
- B: 5+1=6, but B is already explored.
- G: 5+2=7

The new path to G costs 7, which is cheaper than the existing frontier cost of 9.

Therefore, replace G:9(C) with G:7(D).

Priority queue:

[G:7(D)]

Explored set: {S, A, C, B, D}

Iteration 6

Remove G with cost 7.

The goal test succeeds.

Stop the search.

Priority queue:

[]

Explored set: {S, A, C, B, D, G}

Completed UCS Table

Iteration	Node Expanded	g(n)g(n)	Priority Queue After Expansion
1	S	0	[A:1(S), B:4(S)]
2	A	1	[C:3(A), B:4(S), D:6(A)]
3	C	3	[B:4(S), D:6(A), G:9(C)]
4	B	4	[D:5(B), G:9(C)]
5	D	5	[G:7(D)]
6	G	7	[]

UCS Solution Path

Using the parent links:

- G's parent is D.
- D's parent is B.
- B's parent is S.

Therefore, the path is:

S→B→D→G

Path cost:

4+1+2=7

Result: UCS finds a path with cost 7, which is cheaper than the path found by BFS and DFS.

(d) Comparison of Search Strategies

Strategy	Path Found	Path Cost	Nodes Removed from Frontier	Optimal Here?
BFS	S→A→C→G	9	6	No
DFS	S→A→C→G	9	4	No
UCS	S→B→D→G	7	6	Yes

Explanation

BFS: Finds the path with the fewest edges. Both available routes to G have three edges, but BFS reaches G through the A–C branch first due to alphabetical ordering. It does not consider edge costs when searching.

DFS: Follows the alphabetically first branch deeply and reaches G through A and C. It removes fewer nodes because it reaches the goal before exploring the remaining branches.

UCS: Expands nodes according to cumulative path cost. It finds the route through B and D, with a total cost of 7, which is less than the cost of 9 for the other route.

(e) Interpretation

1. Why Did UCS Remove as Many Nodes as BFS Yet Find a Cheaper Path?

Both UCS and BFS removed 6 nodes from the frontier, including the goal node. However, they use different selection criteria.

- **BFS** selects nodes based on depth and ignores edge costs during search.
- **UCS** selects nodes based on the lowest cumulative path cost.

At iteration 4, UCS discovers a cheaper route to D through B:

S→B→DS

The cost is:

4+1=5

This replaces the previous frontier entry for D with cost 6.

At iteration 5, UCS discovers a cheaper route to G through D:

S→B→D→G

Its cost is:

4+1+2=7

Therefore, UCS finds a cheaper solution even though it removes the same number of nodes as BFS.

2. Why Did DFS Need the Fewest Nodes, and Why Cannot It Be Trusted in General?

DFS removes only 4 nodes: S, A, C, and G.

It reaches the goal quickly by following the first branch in alphabetical order.

However, DFS cannot guarantee the lowest-cost solution because it does not consider edge costs.

For example, DFS finds a path with cost 9, while UCS finds a path with cost 7.

In other graphs, DFS may follow a very long path before reaching the goal. In infinite-depth search spaces, it may fail to find a solution even when one exists.

3. Why Is an Explored Set Needed in Graph Search?

An explored set prevents the algorithm from repeatedly expanding nodes that have already been visited.

Consider iteration 3 of BFS, when node B is dequeued.

B's neighbours are D and S:

- D is already in the queue, so it must not be added again.
- S is already in the explored set, so it must not be revisited.

Without duplicate-state checking, the algorithm could add the same nodes repeatedly, waste memory, and perform unnecessary expansions. In graphs containing cycles, this could lead to repeated traversal.

Therefore, an explored set helps graph-search algorithms avoid unnecessary repeated work and ensures systematic exploration.

Final Conclusion

This problem demonstrates the differences between three uninformed search strategies:

- **BFS** explores level by level and finds a solution with the fewest edges.
- **DFS** explores one branch deeply and can reach a goal with fewer node removals, but does not guarantee an optimal path.
- **UCS** expands the lowest-cost frontier node and finds the minimum-cost solution when action costs are non-negative.

For the given graph:

BFS Cost=9 DFS Cost=9 UCS Cost=7

Therefore, Uniform-Cost Search is the optimal strategy for this weighted graph.

Question 4: Search with Partial Information – Belief States

Given Information

A vacuum-cleaner agent operates in a world containing two squares: Left and Right. Each square can be Clean (C) or Dirty (D).

The agent can perform three actions:

- **Left:** Move to the Left square.
- **Right:** Move to the Right square.
- **Suck:** Clean the square where the agent is currently located.

If the agent is already in the specified square, the Left or Right action leaves it in the same location. The Suck action cleans the current square.

Physical States

State	Description
S1	(L, D, D)
S2	(R, D, D)
S3	(L, D, C)
S4	(R, D, C)

S5	(L, C, D)
S6	(R, C, D)
S7	(L, C, C)
S8	(R, C, C)

A belief state is a set of physical states in which the agent believes the actual world might be.

(a) Formulation of the Problem Over Belief States

Since the agent initially has no sensors and does not know its location or the dirt status, it must reason about all possible physical states.

1. Initial Belief State

Initially, all eight physical states are possible.

B0= {S1, S2, S3, S4, S5, S6, S7, S8}

2. Actions

The available actions are:

A= {Left, Right, Suck}

Each action transforms every physical state in the current belief state into its corresponding successor state. The resulting belief state is the union of all these successors.

3. Goal Test

The goal is to have both squares clean, regardless of the agent's location.

Therefore, the goal belief state must contain only physical states in which both squares are clean:

B⊆ {S7, S8}

A belief state satisfies the goal test if every physical state in that belief state has both squares clean.

4. Path Cost

Assume each action has a cost of 1.

Path Cost=Number of actions performed

For example, the sequence [Right, Suck, Left, Suck] has a path cost of 4.

(b) Apply the Action Sequence [Right, Suck, Left, Suck]

In the sensorless environment, the agent cannot observe its location or the dirt status of either square.

Therefore, after every action, it must consider all possible successor states.

Initial Belief State

B0={S1, S2, S3, S4, S5, S6, S7, S8}

Step 1: Perform Right

The Right action moves the agent to the Right square. If it is already there, it remains there.

The transitions are:

Initial State	**Successor State**
S1 →	S2
S2 →	S2
S3 →	S4
S4 →	S4
S5 →	S6
S6 →	S6
S7 →	S8
S8 →	S8

Taking the union of all successor states:

B1= {S2, S4, S6, S8}

Belief state after Step 1:

B1= {S2, S4, S6, S8}

Step 2: Perform Suck

The Suck action cleans the Right square.

The transitions are:

Initial State	Successor State
S2 →	S2
S4 →	S4
S6 →	S8
S8 →	S8

Explanation:

- S2 represents (R, D, D). Cleaning Right leaves (R, D, C), which is S4, not S2.
- S4 represents (R, D, C). Cleaning Right leaves it unchanged.
- S6 represents (R, C, D). Cleaning Right produces (R, C, C), which is S8.
- S8 represents (R, C, C). Cleaning Right leaves it unchanged.

Correct successor calculation:

- S2 → S4
- S4 → S4
- S6 → S8
- S8 → S8

Therefore,

B2= {S4, S8}

Belief state after Step 2:

B2= {S4, S8}

Step 3: Perform Left

The Left action moves the agent to the Left square.

The transitions are:

Initial State	Successor State
S4 →	S3
S8 →	S7

Therefore,

B3= {S3, S7}

Belief state after Step 3:

B3= {S3, S7}

Step 4: Perform Suck

The agent is now in the Left square. The Suck action cleans the Left square.

The transitions are:

Initial State	Successor State
S3 →	S7
S7 →	S7

Explanation:

- S3 represents (L, D, C). Cleaning Left produces (L, C, C), which is S7.
- S7 represents (L, C, C). Cleaning Left leaves it unchanged.

Therefore,

B4= {S7}

Belief state after Step 4:

B4= {S7}

Completed Table for Part (b)

Step	Action	Belief State After Action
0	Start	{S1, S2, S3, S4, S5, S6, S7, S8}

1	Right	{S2, S4, S6, S8}
2	Suck	{S4, S8}
3	Left	{S3, S7}
4	Suck	{S7}

Conclusion: After performing the four actions, the belief state becomes {S7}. This means the agent knows that both squares are clean and that it is in the Left square, even though it received no percepts during execution.

(c) Belief-State Updates with a Local Sensor

Now the agent has a local sensor that reports:

[Location, Status of Current Square]

The agent begins with the same initial belief state:

B0= {S1, S2, S3, S4, S5, S6, S7, S8}

Unlike the sensorless case, the agent can use percepts to eliminate physical states inconsistent with its observations.

Important Definitions

Prediction: Apply the selected action to every state in the current belief state and combine the resulting states.

$B' = \text{Predict}_{f_0}(B, a)$

Update: Keep only those predicted states consistent with the received percept.

$B'' = \{s \in B' : s \text{ is consistent with the percept}\}$

Step 0: Initial Percept [Left, Dirty]

The initial belief state is:

B0= {S1, S2, S3, S4, S5, S6, S7, S8}

The percept is:

[Left, Dirty]

The agent must be in the Left square, and the Left square must be Dirty.

The consistent states are:

- S1 = (L, D, D)
- S3 = (L, D, C)

Therefore, the updated belief state is:

B0′= {S1, S3}

Step 1: Perform Suck

The agent performs Suck in the Left square.

The predicted transitions are:

Initial State	**Successor State**
S1 = (L, D, D)	S5 = (L, C, D)
S3 = (L, D, C)	S7 = (L, C, C)

The predicted belief state is:

B1pred= {S5, S7}

The agent then receives the percept:

[Left, Clean]

Both S5 and S7 have the agent in Left and the Left square clean.

Therefore, the updated belief state is:

B1updated= {S5, S7}

Step 2: Perform Right

The agent performs Right from the current belief state {S5, S7}.

The predicted transitions are:

Initial State	**Successor State**
S5 = (L, C, D)	S6 = (R, C, D)

S7 = (L, C, C)	S8 = (R, C, C)

The predicted belief state is:

B2pred= {S6, S8}

The agent receives the percept:

[Right, Dirty]

The agent is in Right, and the Right square is Dirty.

- S6 = (R, C, D) is consistent.
- S8 = (R, C, C) is inconsistent because its Right square is clean.

Therefore, the updated belief state is:

B2updated= {S6}

Completed Table for Part (c)

Step	**Action**	**After Action (Predicted)**	**Percept**	**After Percept (Updated)**
0	Start	{S1, S2, S3, S4, S5, S6, S7, S8}	[Left, Dirty]	{S1, S3}
1	Suck	{S5, S7}	[Left, Clean]	{S5, S7}
2	Right	{S6, S8}	[Right, Dirty]	{S6}

Conclusion: After receiving the percept [Right, Dirty], the belief state contains only S6. The agent now knows that it is in the Right square, the Right square is Dirty, and the Left square is Clean.

(d) Number of Possible Belief States

There are 8 physical states in the vacuum-cleaner world.

A belief state is any subset of the set of physical states.

If there are nn physical states, the number of possible belief states is:

2n

Here, n=8.

Therefore,

2^8=256

Number of possible belief states=256

This includes the empty set, which represents an impossible or inconsistent belief state. If only non-empty belief states are counted, there are:

256−1=255

Why Is Belief-State Search Harder Than Physical-State Search?

Searching over physical states involves only 8 possible states in this problem.

However, searching over belief states involves up to 256 possible subsets of those physical states.

This makes the problem harder because:

1. The number of possible search states increases significantly.
2. Each action may transform multiple physical states into a new belief state.
3. With sensors, percepts must be used to filter the predicted belief state.
4. More belief states may need to be explored before finding a solution.

Thus, belief-state search can require considerably more memory and computation than searching over fully known physical states.

(e) Interpretation

1. Why Do the Percepts in Part (c) Shrink the Belief State?

A percept provides information about the actual environment. Any physical state inconsistent with the percept can be removed from the belief state.

For example, the initial belief state contains all eight states:

B0= {S1, S2, S3, S4, S5, S6, S7, S8}

When the agent receives [Left, Dirty], it knows that:

- Its location is Left.

- The Left square is Dirty.

Only S1 and S3 satisfy these conditions.

Therefore, the belief state shrinks to:

{S1, S3}

Later, the percept [Right, Dirty] eliminates S8 because the Right square is clean in S8.

The belief state consequently shrinks from {S6, S8} to {S6}.

Thus, percepts reduce uncertainty by eliminating physical states that contradict the agent's observations.

2. How Does This Differ from the Sensorless Case in Part (b)?

Sensorless Case (Part b)	**Percept-Based Case (Part c)**
The agent has no sensors.	The agent has a local sensor.
The agent cannot directly observe its location or dirt status.	The agent receives percepts describing its location and the current square's status.
Belief states change only through action predictions.	Belief states change through both action predictions and percept updates.
The agent must consider all physical states consistent with its initial knowledge and actions.	The agent can eliminate states inconsistent with its percepts.
Uncertainty may remain after actions.	Percepts can reduce uncertainty and sometimes identify one physical state.

For example, in Part (b), after the action sequence [Right, Suck, Left, Suck], the belief state becomes {S7}.

In Part (c), after the percept [Right, Dirty], the belief state becomes {S6}.

The key difference is that the sensorless agent relies only on its action history, whereas the percept-based agent also uses new information from its sensors.

3. Why Can the Sensorless Plan Still Be Called Rational?

A rational agent selects actions that maximize its expected performance based on its available information and the performance measure.

Even without sensors, the agent can reason about all possible physical states and select a plan that works across those possibilities.

In Part (b), the action sequence is:

[Right, Suck, Left, Suck]

It transforms the initial belief state into:

B4={S7}

State S7 represents:

(L, C, C)

Therefore, both squares are clean, and the agent knows this must be the final state.

The plan is rational because it achieves the goal despite the agent's initial uncertainty. The agent does not need to know the exact initial physical state if it can find an action sequence that guarantees the goal for every state in its initial belief state.

Conclusion: Rationality does not require complete information. A sensorless agent can act rationally by selecting a plan that achieves its goal across all possible states it considers to be the actual state of the environment.

Overall conclusion: Belief-state search enables an intelligent agent to reason under uncertainty. Without sensors, the agent predicts the possible outcomes of its actions. With sensors, it can also update its beliefs using percepts, reducing uncertainty and improving its understanding of the environment.